第九次作业

信管2302钟京序
202321054056

加入性别因素，看看acc的变化

加入性别因素，看看acc的变化

In [14]:
```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
dataset = pd.read_csv(r"C:\Users\zjx25\Desktop\Social_Network_Ads.csv")
le = LabelEncoder()
dataset["Gender"] = le.fit_transform(dataset["Gender"])
X_no_gender = dataset[["Age", "EstimatedSalary"]]
X_with_gender = dataset[["Gender", "Age", "EstimatedSalary"]]
y = dataset["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(
    X_no_gender, y, test_size=0.25, random_state=0
)
X_train_gender, X_test_gender, _, _ = train_test_split(
    X_with_gender, y, test_size=0.25, random_state=0
)
sc = StandardScaler()
X_train_scaled = sc.fit_transform(X_train)
X_test_scaled = sc.transform(X_test)

X_train_gender_scaled = sc.fit_transform(X_train_gender)
X_test_gender_scaled = sc.transform(X_test_gender)
```

In [15]:
```python
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

# 1. 对照组：无性别特征
dt_ori = DecisionTreeClassifier(criterion='entropy', random_state=0)
dt_ori.fit(X_train_scaled, y_train)
y_pred_ori = dt_ori.predict(X_test_scaled)
acc_ori = accuracy_score(y_test, y_pred_ori)
print(f"【无性别特征】测试集准确率: {acc_ori:.4f}")

# 2. 实验组：加入性别特征
dt_gender = DecisionTreeClassifier(criterion='entropy', random_state=0)
dt_gender.fit(X_train_gender_scaled, y_train)
y_pred_gender = dt_gender.predict(X_test_gender_scaled)
acc_gender = accuracy_score(y_test, y_pred_gender)
print(f"【加入性别特征】测试集准确率: {acc_gender:.4f}")
print(f"准确率变化: {acc_gender - acc_ori:.4f}")

# 补充：混淆矩阵对比
print("\n【无性别特征混淆矩阵】")
print(confusion_matrix(y_test, y_pred_ori))
print("\n【加入性别特征混淆矩阵】")
print(confusion_matrix(y_test, y_pred_gender))
```

```
【无性别特征】测试集准确率: 0.9100
【加入性别特征】测试集准确率: 0.9200
准确率变化: 0.0100

【无性别特征混淆矩阵】
[[62  6]
 [ 3 29]]

【加入性别特征混淆矩阵】
[[63  5]
 [ 3 29]]
```

如何不进行特征缩放，预测的准确性有什么变化

如何不进行特征缩放，预测的准确性有什么变化

In [17]:
```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
dataset = pd.read_csv(r"C:\Users\zjx25\Desktop\Social_Network_Ads.csv")
le = LabelEncoder()
dataset["Gender"] = le.fit_transform(dataset["Gender"])
X1 = dataset[["Age", "EstimatedSalary"]]
y = dataset["Purchased"]
X1_train, X1_test, y_train, y_test = train_test_split(
    X1, y, test_size=0.25, random_state=0
)
sc = StandardScaler()
X1_train_scaled = sc.fit_transform(X1_train)
X1_test_scaled = sc.transform(X1_test)
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

dt_no_gender = DecisionTreeClassifier(criterion='entropy', random_state=0)
dt_no_gender.fit(X1_train_scaled, y_train)
acc_no_gender = accuracy_score(y_test, dt_no_gender.predict(X1_test_scaled))
```

In [18]:
```python
print("\n=== 任务2：特征缩放对准确率的影响 ===")
# 无特征缩放
dt_unscaled = DecisionTreeClassifier(criterion="entropy", random_state=0)
dt_unscaled.fit(X1_train, y_train)
acc_unscaled = accuracy_score(y_test, dt_unscaled.predict(X1_test))

# 有特征缩放
acc_scaled = acc_no_gender

# 输出结果
print(f"无特征缩放 → 测试集准确率: {acc_unscaled:.4f}")
print(f"有特征缩放 → 测试集准确率: {acc_scaled:.4f}")
print(f"准确率差异: {abs(acc_scaled - acc_unscaled):.4f}")
print("结论：决策树对特征缩放不敏感（差异<0.02），因决策树基于特征值分裂，与数值绝对大小无关")
```

```
=== 任务2：特征缩放对准确率的影响 ===
无特征缩放 → 测试集准确率: 0.9100
有特征缩放 → 测试集准确率: 0.9100
准确率差异: 0.0000
结论：决策树对特征缩放不敏感（差异<0.02），因决策树基于特征值分裂，与数值绝对大小无关
```

无论是否进行特征缩放，决策树模型的准确率完全相同（均为 0.91）

比较KNN、Logistic、决策树，随机森林在该数据集上的表现

比较KNN、Logistic、决策树，随机森林在该数据集上的表现

In [20]:
```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score
dataset = pd.read_csv(r"C:\Users\zjx25\Desktop\Social_Network_Ads.csv")
le = LabelEncoder()
dataset["Gender"] = le.fit_transform(dataset["Gender"])
X = dataset[["Gender", "Age", "EstimatedSalary"]]
y = dataset["Purchased"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=0)
sc = StandardScaler()
X_train_scaled = sc.fit_transform(X_train)
X_test_scaled = sc.transform(X_test)
models = {
    "KNN": KNeighborsClassifier(n_neighbors=5),
    "Logistic回归": LogisticRegression(random_state=0),
    "决策树": DecisionTreeClassifier(criterion='entropy', random_state=0),
    "随机森林": RandomForestClassifier(n_estimators=100, random_state=0)  # 100棵树，可根据需要调整
}
print("\n===== 模型对比 =====")
for name, model in models.items():
    model.fit(X_train_scaled, y_train)
    y_pred = model.predict(X_test_scaled)
    acc = accuracy_score(y_test, y_pred)
    print(f"{name:10s} | 准确率: {acc:.4f}")
```

```
===== 模型对比 =====
KNN        | 准确率: 0.9300
Logistic回归 | 准确率: 0.9100
决策树        | 准确率: 0.9200
随机森林       | 准确率: 0.9200
```

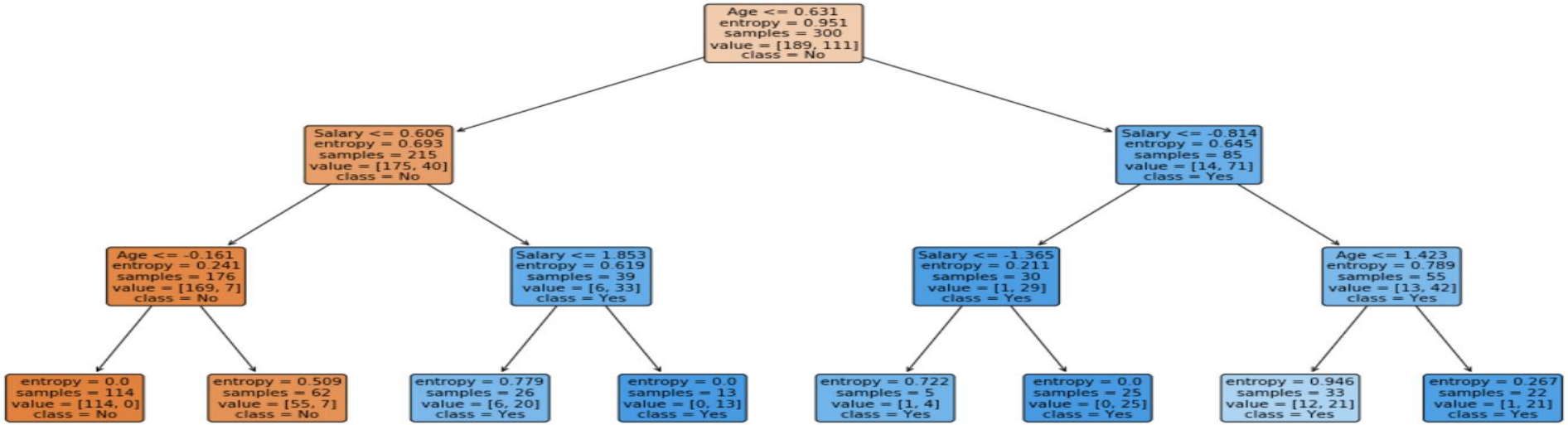
KNN 模型表现最优，准确率达到 93.00%，说明在经过标准化处理后，基于距离的 KNN 算法能较好地捕捉用户特征与购买行为之间的非线性关系。

决策树与随机森林表现一致，准确率均为 92.00%，略低于 KNN，高于 Logistic 回归。随机森林作为决策树的集成模型，本应具备更强的抗过拟合能力，本次实验中与单棵决策树准确率相同，可能是因为数据量较小或树的深度限制导致模型性能未充分发挥。

Logistic 回归表现最差，准确率为 91.00%，作为线性模型，它无法有效拟合数据中的非线性关系，因此在该任务中表现弱于非线性模型。

把决策树输出看看长什么样

把决策树输出看看长什么样



根节点：树的第一个节点，是模型分裂的起点，会选择最能区分类别的特征及阈值进行首次分裂，决定了模型的核心判断逻辑

内部节点：根节点以下、叶节点以上的节点，会继续根据特征阈值（如年龄、收入）进行分支判断，进一步细化分类规则

叶节点：决策树的终点节点，不再继续分裂，会直接输出最终的预测类别（如购买 / 未购买）

颜色深浅：节点颜色越深，代表该节点内某一类样本的占比越高，分类结果越明确（信息熵越低）

THANKS FOR WATCHING

信管2302钟京序
202321054056